

TrustRAG

Make retrieval a security decision.

Authenticity, integrity, trust and authorization
before private documents reach an LLM.

*“The most semantically relevant document
is not necessarily the safest document.”*

**VERIFY
FILTER
THEN
GENERATE**

Similarity is not a security guarantee

A naive RAG pipeline can retrieve relevant text that should never become answer context.

Knowledge poisoning

Misleading document content can distort a generated answer.

Prompt injection

Instructions embedded in retrieved text can compete with the intended task.

Tampering & access

Modified files or restricted documents can cross an unprotected retrieval path.

Relevant is necessary. Eligible comes first.

Secure retrieval combines hard policy checks with trust-aware ranking.

01 Authenticity & integrity
Check current file evidence and signed bytes.

02 Authorization & content
Enforce classification, quarantine and content checks.

03 Trust-aware relevance
Rank Trusted before Suspicious; then use distance.

**Only eligible
context
reaches
generation**

Provenance records history;
it is not a separate access gate.

Local retrieval, external generation

Security checks are integrated into both ingestion and query processing.

React interface

7 authenticated pages



FastAPI backend

JWT • RBAC • orchestration



Secure RAG core

Ingest • verify • retrieve

Local data & models

EmbeddingGemma-300M • 768 dimensions • ChromaDB
Files + metadata • Ed25519 keys • provenance • SQLite audit

OpenRouter LLM

The prompt and eligible
document
context leave the local system.

Build security evidence before indexing

PDF, DOCX and TXT enter an authenticated, classification-aware ingestion path.

1 Validate

Type, size and clearance;
server-generated document ID.

2 Verify

SHA-256 file digest;
Ed25519 when signed.

3 Extract & scan

Read text; scan for injection
and poisoning patterns.

4 Assign trust

Trusted, Suspicious
or Quarantined.

5 Chunk & embed

512-character target, 64 overlap;
local 768-dimensional vectors.

6 Persist evidence

File/chunk digests, classification,
trust metadata and provenance.

Three categories. Explicit outcomes.

Trust is a policy classification—not a calibrated probability of safety.

Trusted

Valid signature
+ clean scans

Eligible; ranked first

Suspicious

Unsigned
+ clean scans

Eligible; ranked lower

Quarantined

Invalid signature OR
positive content scan

Blocked from secure context

Recheck evidence at retrieval time

Ingestion-time labels alone are not enough: stored content can change.

01 Retrieve candidates

Embed query and fetch up to $3 \times$ top-K from ChromaDB.

02 Validate document evidence

Check authoritative records, current file digests and signatures.

03 Enforce eligibility

Apply classification and quarantine; verify chunk digests; rescan content.

04 Rank & generate

Trusted first, then distance. Send eligible context to the LLM.

Changed evidence can persist quarantine across metadata, vectors and provenance.

Access control spans the full workflow

Authorization is enforced server-side, not only by hiding controls in the interface.

Four role profiles

Employee

HR

IT_Security

Admin

Authenticated access

Data, dashboard, status, evaluation and query APIs require JWTs.

Classification enforcement

Uploads, listings, details and secure retrieval respect clearance.

Privileged operations

Unfiltered baseline and audit event access: Admin / IT_Security.

Expose decisions—not just answers

A seven-page React interface connects document management, query context and security telemetry.

Dashboard

Documents

Chat

Evaluation

Audit Log

Knowledge Base

Status

Upload / query event traces

Request ID, outcome, stages and duration
Privileged event filters and expandable traces
Current call sites omit prompts, text and secrets

SQLite audit and JSON provenance are mutable;
neither is cryptographically tamper-evident.

Implemented controls, bounded evidence

Reported verification from the project handoff on 17 September 2026—not a new run for this deck.

67

**automated
tests passed**

1 deprecation warning
Frontend build + lint
passed

Four synthetic component scenarios

Cryptographic tampering

Indirect prompt injection

Knowledge poisoning

RBAC authorization

**Not an end-to-end attack-success benchmark.
Suite runtime is not measured secure-RAG latency overhead.**

What the prototype does not prove

Security claims must stay within the threat model and the evidence collected.

Heuristic detection

Regex checks can miss adversarial content or flag benign text.

Held-out corpus; false-positive / false-negative analysis.

Trusted storage & operators

Administrators, metadata storage and configured keys remain trusted.

Key lifecycle, reconciliation and stronger audit integrity.

External processing

Eligible document context goes to OpenRouter.

Explicit egress policy and approved data handling.

Unmeasured end-to-end behavior

Retrieval quality, answer quality and security overhead remain open.

Controlled baseline comparison with measured outcomes.

Production hardening also needs TLS, request/rate limits, quotas, identity lifecycle and backups.

Trust before generation.

TrustRAG turns retrieval from a similarity lookup into a verified, policy-aware selection process.

Verify evidence

Enforce access

Filter & rank

Working academic prototype.

Broader security and utility claims still require measured experiments.